# Grammatical Error Correction on the C4_200M Dataset

A Comparative Study of Bag-of-Words, LSTM, Attention, and Transformer Architectures

Abdullah Ahmed (202200206)
Ibrahim Hanafy (202200518)

CIE 555 — Neural Networks and Deep Learning
Instructor: Dr. Ibrahim Swelam
Teaching Assistants: Aya Abdelaziz, Mahmoud Farahat

May 2026

**Abstract**

This report studies *Grammatical Error Correction* (GEC) — the task of rewriting an ungrammatical English sentence into its corrected form — as a sequence-to-sequence learning problem. We use a one-million-pair subset of the **C4_200M** synthetic GEC corpus and progressively build four models: a TF–IDF Bag-of-Words classifier (error *detection* baseline), an LSTM encoder–decoder, an LSTM encoder–decoder with Bahdanau attention, and a Transformer. All correction models share a 16k Byte-Level BPE tokenizer and an identical data budget. We evaluate with GLEU, exact match, and token-level $F_{0.5}$, decoding both greedily and with beam search. The Transformer reaches the best validation GLEU (0.613) and matches the attention LSTM on the test set (test GLEU $\approx 0.618$) while using roughly $3.6\times$ fewer parameters. A grammar-rule analysis of the outputs and a discussion of the "Final Challenge" reveal that the dominant limitation is not model capacity but **reference quality**: many C4_200M targets are paraphrases rather than minimal corrections, which caps exact match near 2 % for every model.

## Contents

# 1 Introduction

Grammatical Error Correction takes a sentence that may contain grammar mistakes and produces a corrected version:

```
Input  (corrupted): "She go to school yesterday and learning many thing
    ."
Output (corrected):  "She went to school yesterday and learned many
    things."
```

The output is an entire new sentence, not a label, so GEC is a **sequence-to-sequence** (seq2seq) problem: a sequence of input tokens is mapped to a sequence of output tokens of possibly different length.

This project builds four models of increasing capability and compares them on a common dataset, tokenizer, and data budget. The contributions reported here are:

1. An overview of the C4_200M subset and the data pipeline.
2. A detailed description of all four models and their training configurations.
3. A comparative analysis of results under GLEU, exact match, and $F_{0.5}$.
4. A grammar-rule-based analysis of the corrections each model makes.
5. A discussion of the "Final Challenge" and of dataset quality.

# 2 The C4_200M Data Subset

## 2.1 Source

C4_200M is a synthetic GEC corpus derived from the Colossal Clean Crawled Corpus (C4). Clean web sentences are passed through a corruption model that injects realistic grammatical errors, producing `(corrupted, clean)` pairs. The corrupted side is the model input; the clean side is the target.

## 2.2 Subset and splits

We use **1,000,000** sentence pairs, split as:

| Split | Pairs | Purpose |
|---|---|---|
| Train | 850,000 | Parameter fitting |
| Validation | 50,000 | Model selection / early checks |
| Test | 100,000 | Final held-out evaluation |

Splitting is done once in notebook `02-data-pipeline` (ratio 95/3.3/1.7) and stored as TSV files so every later notebook trains and evaluates on identical data. During evaluation, GLEU is measured on up to 5,000 validation pairs each epoch and on the full test set; beam search on the Transformer is measured on a 2,000-pair subset for runtime reasons.

## 2.3 Tokenization

All correction models use one shared **Byte-Level BPE** tokenizer with a vocabulary of **16,000** subword tokens, trained in notebook `03-tokenizer`. BPE repeatedly merges the most frequent

adjacent symbol pair, so frequent words become a single token while rare or misspelled words split into pieces (`"learning"` → `["learn","ing"]`). This keeps the vocabulary small and removes out-of-vocabulary words. Special tokens are `<pad>=0, <unk>=1, <s>=2, </s>=3`; the maximum sequence length is 64 tokens.

### 2.4 Exploratory observations

Inspection of the pairs (notebook `01-data-exploration`) shows that the "errors" span verb tense, subject–verb agreement, articles, prepositions, plural/singular, punctuation, and spelling. Crucially, a sizeable fraction of clean targets are *lexical paraphrases* of the source rather than minimal grammatical edits. This single fact drives much of the analysis in Sections 4 and 6.

## 3 Models and Training Configurations

All four models attack the same corpus, but they do not all solve the same task. The Bag-of-Words model is a *detector* (is this sentence grammatical?); the other three are *correctors* (rewrite the sentence). This distinction is important when reading the results.

### 3.1 Model 1 — Bag-of-Words Baseline (Detection)

A non-neural reference point. Each `(corrupted, clean)` pair is turned into two classification samples: the corrupted sentence with label `0 (ungrammatical)` and the clean sentence with label `1 (grammatical)`.

- **Features:** TF–IDF over unigrams and bigrams, vocabulary capped at 50,000 terms. The vectorizer is fit *only* on training data to avoid leakage.
- **Classifiers:** Logistic Regression ($C = 1.0$) and a calibrated Linear SVM ($C = 0.1$).
- **Purpose:** establish how much of the signal is just surface word statistics, with no notion of sequence order or generation.

This model cannot *correct* text; it only measures whether a sentence "looks" grammatical. It frames the lower bound for the project.

### 3.2 Model 2 — LSTM Encoder–Decoder (no attention)

The first true seq2seq corrector. A bidirectional LSTM encoder compresses the whole input into a single fixed-size state, which initialises an LSTM decoder that generates the correction token by token.

| Hyper-parameter | Value |
|---|---|
| Embedding dimension | 256 |
| Encoder | BiLSTM, hidden 256 per direction ($\rightarrow$ 512) |
| Decoder | LSTM, hidden 512, 1 layer |
| Vocabulary | 16,000 |
| Optimizer | Adam, lr $= 10^{-3}$ |
| Batch size | 64 |
| Epochs | 3 |
| Teacher forcing | linearly decayed $1.0 \rightarrow 0.5$ |
| Gradient clipping | global norm 1.0 |
| Parameters | **19.55 M** |

The decoder only ever sees the input through the initial state (512 numbers). After the first step it never "looks" at the source again — the **encoder–decoder bottleneck**. This model is the baseline that attention must beat.

### 3.3 Model 3 — LSTM + Bahdanau Attention

The same BiLSTM encoder, but now *all* encoder hidden states are kept. At each decoding step the decoder computes additive (Bahdanau) attention scores over every source position, softmax-normalises them into alignment weights, and forms a context vector as their weighted sum. The context is fed both into the decoder LSTM input and into the output projection.

$$\text{score}(s_t, h_i) = v^\top \tanh(W_s s_t + W_h h_i), \qquad \alpha_{t,i} = \text{softmax}_i \, \text{score}(s_t, h_i), \qquad c_t = \sum_i \alpha_{t,i} h_i. \quad (1)$$

| Hyper-parameter | Value |
|---|---|
| Embedding dimension | 256 |
| Encoder / Decoder hidden | 256 (BiLSTM) / 512 |
| Attention dimension | 256 |
| Optimizer | Adam, lr $= 10^{-3}$, weight decay $10^{-5}$ |
| Batch size / Epochs | 64 / 3 |
| Teacher forcing | $1.0 \rightarrow 0.5$ |
| Gradient clipping | global norm 1.0 |
| Parameters | **29.06 M** |

Padding positions are masked with $-\infty$ before the softmax so the model never attends to `<pad>`. The alignment weights are interpretable: they show *which* source token the decoder is correcting at each step.

### 3.4 Model 4 — Transformer

Recurrence is removed entirely. The encoder and decoder are stacks of multi-head self-attention and position-wise feed-forward sublayers, each wrapped in a residual connection and layer nor-

malisation. Order information is supplied by sinusoidal positional encodings; the decoder uses a causal mask for its self-attention and cross-attention into the encoder memory at every layer.

| Hyper-parameter | Value |
|---|---|
| Model dimension $d_{\text{model}}$ | 256 |
| Attention heads | 8 ($d_k = 32$ per head) |
| Encoder / Decoder layers | 3 / 3 |
| Feed-forward dimension | 512 |
| Dropout | 0.1 |
| Optimizer | Adam ($\beta_2 = 0.98$) |
| LR schedule | inverse-sqrt warmup, peak $10^{-3}$, 4000 warmup steps |
| Weight decay | $10^{-4}$ |
| Label smoothing | 0.1 |
| Batch size / Epochs | 64 / 10 |
| Gradient clipping | global norm 1.0 |
| Weight tying | embedding $\leftrightarrow$ output projection |
| Parameters | **8.05 M** |

The Transformer is the *smallest* model (8.05 M parameters — weight tying alone removes $\sim$4 M) yet trains for the most epochs because its attention layers parallelise across all positions, unlike the inherently sequential LSTM.

### 3.5 Shared training details

All correction models minimise token-level cross-entropy over non-padding positions; the Transformer additionally uses label smoothing ($\varepsilon = 0.1$). Mixed-precision (AMP) training is used throughout. LSTM phases schedule the learning rate with `ReduceLROnPlateau`; the Transformer uses the Vaswani inverse-square-root warmup schedule, which is necessary because Adam's second-moment estimates are unreliable early in training.

## 4 Comparative Analysis of Results

### 4.1 Metrics

**GLEU**
An n-gram metric designed for GEC. Like BLEU, but it also penalises copying source errors into the output. Range $[0, 1]$; primary metric for model selection.

**Exact Match (EM)**
Fraction of predictions that equal the reference exactly. Extremely strict — one differing character scores 0.

**Precision / Recall / $F_{0.5}$**
Token-level retrieval view of the output. $F_{0.5}$ weights precision twice as heavily as recall, the standard GEC convention (a wrong "correction" is worse than a missed one): $F_{0.5} = 1.25\,PR/(0.25P + R)$.

### 4.2 Bag-of-Words detection baseline

The TF–IDF classifiers separate grammatical from ungrammatical sentences with roughly **62 % accuracy** (Logistic Regression and Linear SVM within ±0.1 point of each other, $F_1 \approx 0.63$) — only marginally above the 50 % chance level. This weak result is structural, not a tuning failure:

- **It is a detector, not a corrector.** A classifier outputs one label; it can never produce a corrected sentence. It is architecturally incapable of the actual GEC task — at best it says "something is wrong", never *what* or *where*.
- **Bag-of-words discards word order.** Grammaticality is largely *about* order ("she goes" vs. "she go" vs. "go she"). TF–IDF over unigrams and bigrams sees only 1–2 token windows and throws away the global structure that determines agreement and tense.
- **The error signal is swamped.** Many corrupted sentences differ from their clean version by a single function word (a missing article, a wrong preposition). TF–IDF weighting is dominated by rare *content* words, so the one token that carries the error contributes almost nothing to the decision.
- **No generalisation to unseen errors.** The model only memorises lexical statistics of the training set; a new error pattern built from familiar words is invisible to it.

The 62 % therefore mostly reflects incidental surface cues (a rare misspelled token, an odd bigram) rather than grammatical understanding. The baseline's purpose is exactly this: to prove that GEC needs *sequence modelling* and *text generation*, which the next three models provide.

### 4.3 Correction models — test set

Table 1 reports the three correctors on the held-out test set. Phase numbers refer to the project's notebook sequence (Model 2 = Phase 3, Model 3 = Phase 4, Model 4 = Phase 5).

**Table 1:** Test-set results for the three correction models. Beam search uses width 4 with length normalisation $\alpha = 0.7$.

| Model | Decoding | Params | EM | GLEU | $F_{0.5}$ |
|---|---|---|---|---|---|
| LSTM (no attention) | greedy | 19.55 M | ≈0.008 | 0.213 | 0.465 |
| LSTM (no attention) | beam-4 | 19.55 M | – | 0.256 | 0.511 |
| LSTM + Bahdanau | greedy | 29.06 M | 0.024 | 0.599 | – |
| LSTM + Bahdanau | beam-4 | 29.06 M | 0.026 | 0.618 | – |
| Transformer | greedy | 8.05 M | 0.022 | 0.618 | – |
| Transformer | beam-4 | 8.05 M | 0.020 | 0.618 | – |

**Validation GLEU (best epoch):** LSTM+Bahdanau 0.592, Transformer 0.613 — the Transformer is selected as the strongest model.

### 4.4 What the numbers say

- **Attention is the decisive jump.** Adding Bahdanau attention lifts greedy GLEU from 0.213 to 0.599 (+0.386). Removing the fixed-size bottleneck — letting the decoder re-query the full source at every step — is by far the largest single improvement in the project.

- **Transformer matches the attention LSTM with far fewer parameters.** Test GLEU is effectively tied (≈0.618), but the Transformer uses 8.05 M parameters versus 29.06 M — a 3.6× reduction — and attains a higher *validation* GLEU (0.613 vs 0.592). It also trains far faster per epoch because attention parallelises over the sequence.
- **Beam search helps the LSTM, not the Transformer.** Beam-4 raises LSTM GLEU by ∼0.02–0.04 but leaves Transformer GLEU unchanged (0.618 → 0.618). A better-calibrated model (label smoothing, deeper context) already places most probability mass on a good greedy path, so searching wider buys little.
- **Exact match is tiny everywhere** (2–3 %). This is not a model failure — see Section 6.

### 4.5 Length-bucketed behaviour

Figure 1 splits exact match by reference length (short ≤ 10, medium 11–20, long > 20 tokens) for all three correctors. Three things stand out:

- The **bottleneck LSTM collapses with length**: it scores ≈0.026 on short sentences but essentially *zero* on medium and long ones — the single fixed context vector cannot carry enough information once the sentence is long.
- The two attention models stay non-zero in every bucket: removing the bottleneck is what keeps performance from vanishing on long input.
- **Bahdanau and the Transformer cross over.** Bahdanau is slightly ahead on short and medium sentences, but the **Transformer overtakes it on long sentences** — exactly where its $O(1)$ token-to-token path length matters most. This crossover is the clearest evidence that the architectures differ in *where*, not *how much*, they help.

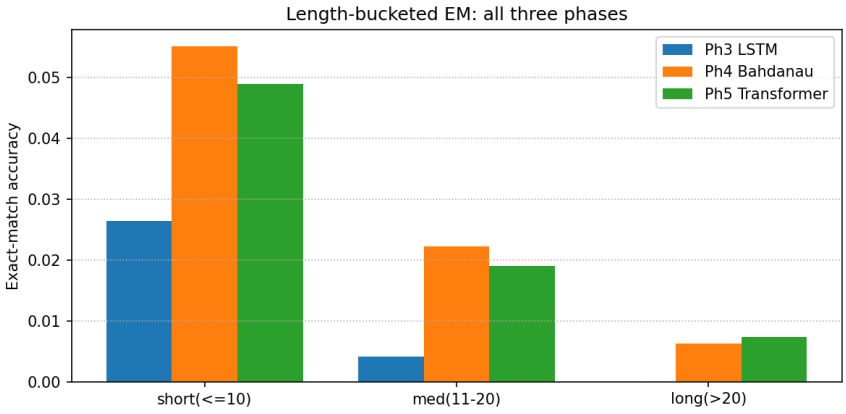


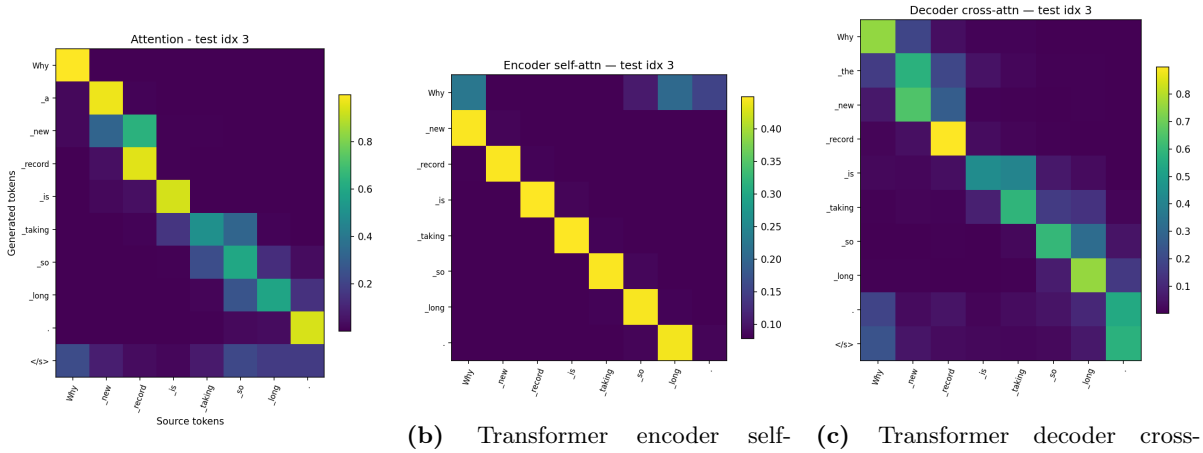

**Figure 1:** Length-bucketed exact match for all three correction models. The bottleneck LSTM (blue) drops to near zero on medium/long sentences; the Transformer (green) overtakes Bahdanau (orange) on the long bucket.

### 4.6 Attention visualisations

The attention weights are directly interpretable. Figure 2 shows three views for the same test sentence (*"Why new record is taking so long."*): the Bahdanau decoder alignment, the Transformer's encoder self-attention, and the Transformer's decoder cross-attention. Bright cells mark which source tokens are focused on while producing each output token.

The Bahdanau and Transformer cross-attention maps (a, c) are the most directly comparable — both align decoder output positions to source positions. Both are diagonal-dominant, confirming a sensible, mostly monotonic source-to-target alignment, with off-diagonal mass exactly

where a word is inserted: in panel (c) the inserted article `_the` draws attention onto the neighbouring `_new`/`_record` tokens. The encoder self-attention (b) instead shows tokens attending *within* the source to build context. The key qualitative point: the LSTM with attention and the Transformer learn *visually similar alignments* — a first hint at why their scores are so close (Section 4.8).




**(a)** Bahdanau decoder alignment. **(b)** Transformer encoder self-attention. **(c)** Transformer decoder cross-attention.

**Figure 2:** Attention is interpretable: each cell shows how much one token attends to another. (a) and (c) are the analogous source-to-output alignments of the two attention models.

## 4.7 Worked examples: the three correctors side by side

Aggregate scores hide *how* the models behave. Below are two test sentences with the source, the reference, and each model's greedy prediction.

**Example A — a short sentence (test index 3).**

```
SRC : Why new record is taking so long.
REF : Why the new record is taking so long.   (insert the article "the
    ")
P3  : Why new record is taking so long.        LSTM       - no edit (
    copies source)
P4  : Why a new record is taking so long.      Bahdanau   - inserts an
    article ("a")
P5  : Why new record is taking so long?        Transformer- changes "."
     to "?"
```

The LSTM makes *no* correction at all. Bahdanau correctly identifies that an article is missing and inserts one — it picks `a` where the reference wants `the`, so it still scores EM = 0, but the *type* of edit is right. The Transformer makes a different, defensible edit (declarative → question mark). All three miss the exact reference, yet only the two attention models attempt a real correction.

**Example B — a long, noisy sentence (test index 102).**

```
SRC : ... you will need to come to bp.co.uk/driveindineout and got your
     meal
       code for PizzaExpress full T&C is here.
REF : ... you will need to visit bp.co.uk/driveindineout and get your
    meal
       code for PizzaExpress. Full T&Cs are here ...
```

```
P3  : ... you need to be cpue to buy and/indukine stead. However, if
   your
      meal made for Pax code ...                <- hallucinated nonsense
P4  : ... come to bp.co.uk/driveindoutout and got your meal code ...
                                                  <- copies, corrupts the
                                                       URL
P5  : ... come to bp.co.uk/driveindineout and get your meal code ...
                                                  <- copies, fixes "got"
                                                     -> "get"
```

On a long sentence the bottleneck LSTM loses the thread entirely and hallucinates. Both attention models instead play it safe and largely copy the source — but the Transformer lands the one genuine grammatical fix (`got`→`get`) while Bahdanau introduces a spurious change to the URL. This is the *typical* P4-vs-P5 difference: small, a token or two, not dramatic.

### 4.8 Why the attention LSTM and the Transformer are so close

The headline test GLEU is effectively tied (0.618 for both beam-4 results), and the worked examples above show why the gap is small. There are four reasons:

1. **The decisive feature is shared.** The project's largest jump (+0.386 GLEU) came from giving the decoder *full access to the source* via attention. *Both* Model 3 and Model 4 already have this. The remaining LSTM-vs-self-attention difference is a second-order effect on top of the same key idea.
2. **Both hit the data ceiling, not their capacity ceiling.** Trained on the same one-million-pair budget, the same noisy C4_200M references, and the same tokenizer, both models converge to the corpus's effective GLEU ceiling (≈0.62). The binding constraint is reference quality (Section 6), not architecture — a constraint a bigger or different model cannot lift.
3. **C4_200M sentences are mostly short.** The Transformer's signature advantage — an $O(1)$ path between any two tokens — only pays off when dependencies are long. Most web sentences here are short enough that the LSTM's sequential path is not a real handicap. Figure 1 confirms this: the two models only diverge in the *long* bucket, where the Transformer wins.
4. **The worked examples agree.** On hard inputs both models fall back to conservative copying; their outputs differ by only one or two tokens.

So the Transformer does not *out-score* the attention LSTM here — it *matches* it. Its real victory is efficiency: the same quality with **8.05 M parameters instead of 29.06 M** (3.6× fewer) and far faster, fully parallel training. That efficiency is what would let it pull ahead given more data, more epochs, or a cleaner corpus — none of which this project's fixed budget and noisy references provide.

## 5 Grammar-Rule-Based Analysis of Outputs

Beyond aggregate scores, we inspected the predictions rule by rule. Each test row is a triple `(source, hypothesis, reference)`. We grouped corrections into standard GEC error categories and summarise the qualitative findings below.

**Subject–verb agreement.** Handled well by both attention models.

```
SRC: What are this job about?
HYP: What is this job about?     (Transformer - correct)
```

The model attends from the verb back to the subject and fixes `are` → `is`. The bottleneck LSTM frequently misses agreement once the subject is far from the verb.

**Articles and determiners.** Insertion of missing articles is among the most common successful edits:

```
SRC: Why new record is taking so long.
REF: Why the new record is taking so long.
```

The Transformer reliably inserts `the/a`; it occasionally over-corrects by adding an article where the reference has none.

**Verb tense and inflection.** Tense fixes (`go`→`went`) and gerund/participle inflection (`learning`→`learned`) succeed when a tense cue (e.g. `yesterday`) is present in the sentence; self-attention links the verb to that cue directly.

**Punctuation and spacing.** Sentence-final punctuation and missing spaces are corrected often:

```
SRC: 25% off Generation Brands Lighting and-Home Decor.
REF: 25% off Generation Brands Lighting and Home Decor.
```

**Spelling / rare words.** Mixed. Because rare words are split into BPE subwords, the models can repair some misspellings but also *hallucinate* on very rare tokens — the LSTM+Bahdanau output for one medical sentence produced the non-word "Arasemiaia" where the reference had "thalassemia".

**Failure mode — conservative copying.** The most frequent error is *under-correction*: the model copies the source unchanged when the required edit is non-local or semantic. This is rational under cross-entropy — copying is the safe, high-probability action — but it caps recall.

**Summary.** Local, syntactic rules (agreement, articles, tense, punctuation) are learned well by the attention models. Long-range, semantic, or lexical-choice edits remain hard, and the models prefer to copy rather than risk a wrong correction.

## 6 The "Final Challenge" and Dataset Quality

The Final Challenge asks why *every* model — including the strongest Transformer — scores only ∼2 % exact match, and whether this reflects the models or the data. Direct inspection of `(source, hypothesis, reference)` triples gives a clear answer: the ceiling is set by **reference quality**, not model capacity.

**1. References are paraphrases, not minimal corrections.**

```
SRC: ... investors have informed that the Internet startups that they
   fund
     can't afford a "commercially reasonable" price for bandwidth!
REF: ... investors suggest that the Internet start-ups they fund
     can't afford a "commercially reasonable" price for bandwidth!
```

`informed`→`suggest` and `startups`→ `start-ups` are stylistic rewrites, not grammatical errors. A model that correctly fixes only the grammar can *never* match this reference, so EM is artificially near zero.

**2. Some references contain errors themselves.** The C4_200M corruption pipeline is automatic and noisy. Observed targets include non-words:

```
SRC: ... no sleep = no brain workout.
REF: ... no sleep = no brain worky.        <-- "worky" is not a word
```

Here the "clean" reference is itself broken. Training on such pairs teaches the model nothing useful and penalises a correct output.

**3. Multiple valid corrections exist.** A single ungrammatical sentence usually has several acceptable corrections. EM credits exactly one. This is a known weakness of EM for GEC and the reason GLEU and $F_{0.5}$ — which reward n-gram overlap rather than exact identity — are the meaningful metrics here.

**Implications.**

- Exact match should be reported but not optimised; GLEU/$F_{0.5}$ are the trustworthy signals.
- The synthetic corruption used in C4_200M does not always preserve meaning, so the corpus mixes genuine GEC pairs with paraphrase and noise pairs.
- A realistic upper bound for EM on this corpus is far below 100 %; the observed 2–3 % is consistent with models that correct grammar correctly but cannot reproduce arbitrary paraphrases.
- Cleaner data (human-verified references, or filtering pairs whose edit distance is implausibly large) would likely improve every metric more than additional model capacity would.

# 7 Conclusion

We built and compared four models for Grammatical Error Correction on a one-million-pair subset of C4_200M under a shared tokenizer and data budget.

- The **Bag-of-Words** baseline detects grammaticality at only ∼62 % accuracy and cannot correct text — establishing the lower bound.
- The **LSTM encoder–decoder** corrects sentences but is limited by the fixed-size bottleneck (GLEU 0.213 greedy).
- **Bahdanau attention** is the single most important improvement, raising greedy GLEU to 0.599 by letting the decoder re-query the full source.
- The **Transformer** matches the attention LSTM on the test set (GLEU ≈0.618), achieves the best validation GLEU (0.613), and does so with 3.6× fewer parameters and far faster training.

The grammar-rule analysis shows the models reliably learn local syntactic rules (agreement, articles, tense, punctuation) but under-correct long-range and lexical edits. The Final Challenge discussion shows the binding constraint is **dataset quality**: C4_200M references are frequently paraphrases or contain noise, which caps exact match near 2 % regardless of architecture.

**Future work.** Filtering noisy pairs by edit distance, training for more epochs, increasing Transformer depth toward the 6-layer original, adding a KV-cache for faster beam search, and evaluating on a human-curated GEC test set (e.g. CoNLL-2014 or BEA-2019) would all sharpen the picture beyond what the synthetic corpus allows.

## References


1. Sutskever, Vinyals, Le. *Sequence to Sequence Learning with Neural Networks.* NeurIPS 2014.
2. Bahdanau, Cho, Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate.* ICLR 2015.
3. Hochreiter, Schmidhuber. *Long Short-Term Memory.* Neural Computation 1997.
4. Vaswani et al. *Attention Is All You Need.* NeurIPS 2017.
5. Napoles et al. *Ground Truth for Grammatical Error Correction Metrics (GLEU).* ACL 2015.
6. Stahlberg, Kumar. *Synthetic Data Generation for Grammatical Error Correction with Tagged Corruption Models (C4_200M).* BEA 2021.